

Lecture 5 | Classifiers, Logits, and Membership Inference

Wednesday, September 9, 2026

Instructors: Steven Wu, Giulia Fanti

In this lecture. This lecture finishes the machine-learning introduction from Lecture 4 and puts it to use. We extend the discriminative recipe from regression to classification, sort out two competing notions of *logit*, and meet GPT-2 as a running example whose exposed probability vectors already hint at a privacy problem. That problem gets a name in the second half: *membership inference*, the question of whether a particular record was used to train a model. We set up the threat model, contrast white-box and black-box access, walk through the classic shadow-model attack, and end with a methodological warning about how *not* to measure an attack's success.

1 The Machine Learning Pipeline, Revisited

Recall the pipeline from Lecture 4: a data holder trains a model $\mathcal{M}$, represented by parameters θ , on a dataset D collected (usually) from other users or purchased outright. Lecture 4 left one arrow out of the picture: a *model user* queries the trained model with a new input x and receives a prediction. The whole pipeline is: data $\rightarrow$ training $\rightarrow$ model $\rightarrow$ inference.

The two questions Lecture 4 asked about this pipeline now have names, which the rest of this lecture develops:

1. **Membership.** Can an outsider infer that a particular user's data was in the training set D ?
2. **Attribute inference.** Can an outsider learn about an attribute of a user's data in D — something specific about that individual's record, not just whether it was there at all?

As before, three choices made before and during training determine what the finished model can leak: the training data itself, the *model architecture* (which fixes the function class $\mathcal{M}$ is drawn from — a transformer, a multilayer perceptron, a linear model), and the *loss function* together with its hyperparameters (which specifies how the data is used to pick one function out of that class). The rest of this lecture makes the architecture and loss sides of that recipe concrete for classifiers, then turns to attacking the result.

2 Regression, Made Concrete

Lecture 4's tip-prediction example was stated for general dimension d . Concretely, it is $d = 1$: each input $x_i \in \mathbb{R}$ is a bill amount, each label $y_i \in \mathbb{R}$ is the tip left on it, and $n = 15$ records make up the training set. Row 14 of the table, for instance, is $x_{14} = 14.83$ and $y_{14} = 3.02$.

The model itself is the inner product $\mathcal{M}(x) = w^\top x$, written out as

$$w^\top x = \sum_{i=1}^d x_i w_i,$$

so that fitting the model means choosing the d numbers in $w \in \mathbb{R}^d$ (here, $d = 1$, a single slope) that minimize the mean squared error from Lecture 4. Regression is the smallest possible instance of the architecture/loss/data recipe; classifiers, next, are the same recipe with a different function class and a different loss.

3 Classifiers

A classification dataset pairs each sample $x_i \in \mathbb{R}^d$ with a *one-hot* label $y_i \in \mathbb{R}^\ell$: a vector of zeros with a single 1 marking the true class. With three classes {cat, dog, plane} ($\ell = 3$), a dog is labeled $y = (0, 1, 0)^\top$.

Definition 3.1. A classifier is a model $\mathcal{M}_\theta$, parameterized by θ , whose output on input x is a probability vector $\hat{y} = \mathcal{M}_\theta(x) \in \mathbb{R}^\ell$ over the ℓ classes — for example $\hat{y} = (0.1, 0.9, 0.0)^\top$, assigning probability 0.9 to dog. Training chooses θ so that $\hat{y}$ lines up with the true one-hot label y as often as possible.

The loss that does this alignment is *cross-entropy*:

$$\min_{\theta} -\mathbb{E}[y^\top \log \hat{y}] = \min_{\theta} -\frac{1}{n} \sum_{i=1}^n y_i^\top \log(\mathcal{M}_\theta(x_i)). \quad (1)$$

Because y_i is one-hot, $y_i^\top \log(\hat{y}_i)$ collapses to a single term: if c is the true class, the whole inner product is just $\log(\hat{y}_{i,c})$, the log-probability the model assigned to the right answer. Minimizing (1) therefore does exactly what a classifier should: push as much probability mass as possible onto the true class, on average over the training set. This is the same recipe as regression — an architecture, a loss, and data — with a vector-valued output and a different loss in place of squared error.

4 A Tale of Two Logits

The word *logit* means two different things depending on which literature you are reading, and both meanings matter here.

4.1 The Statistical Logit

Definition 4.1. Let P_{dog} be the probability a model assigns to class dog. The *odds* of dog are $P_{\text{dog}}/(1 - P_{\text{dog}})$, and the *logit* of dog is the log-odds,

$$\text{logit}(P_{\text{dog}}) = \log\left(\frac{P_{\text{dog}}}{1 - P_{\text{dog}}}\right).$$

A logit can take any value in $(-\infty, \infty)$: it goes to $-\infty$ as $P_{\text{dog}} \rightarrow 0$ and to $+\infty$ as $P_{\text{dog}} \rightarrow 1$. Running the definition backward recovers P_{dog} from a logit z : from $e^z = P_{\text{dog}}/(1 - P_{\text{dog}})$,

$$e^z(1 - P_{\text{dog}}) = P_{\text{dog}} \implies P_{\text{dog}}(1 + e^z) = e^z \implies P_{\text{dog}} = \frac{e^z}{1 + e^z},$$

the familiar S-shaped *sigmoid* function of z .

4.2 The AI Logit

Definition 4.2. In a neural network with a softmax output layer, the *logits* are the raw, unnormalized vector $Z \in \mathbb{R}^\ell$ produced by the last layer, before the softmax is applied. The *softmax* transform of Z is

$$\text{softmax}(Z)_k = \hat{y}_k = \frac{e^{Z_k}}{\sum_{j=1}^{\ell} e^{Z_j}}, \quad k = 1, \dots, \ell. \quad (2)$$

Exercise 4.3. Show that $\sum_{k=1}^{\ell} \hat{y}_k = 1$, so that $\hat{y}$ is a genuine probability vector.

This is deliberately *not* the same object as the statistical logit of Section 4.1, even though both go by the same name. One way to see the gap: the statistical logit of P_{dog} is a single number, uniquely determined once P_{dog} is fixed. The AI logit vector Z is not unique at all — adding any constant c to every entry of Z leaves (2) completely unchanged, since the e^c factor cancels between numerator and denominator. “The logits” in a neural network are only ever defined up to a shared additive shift; the statistical logit has no such slack.

5 GPT-2 as a Running Example

We will use the GPT-2 model for your project: an *autoregressive* decoder-only transformer, trained by next-token prediction. Framed as above, next-token prediction is itself a classification problem: given a prefix, predict the next token from the vocabulary. Continuing “Hi, I am ___”, the model might output

$$\hat{y} = \begin{pmatrix} 0.7 \\ 0.2 \\ 0.1 \end{pmatrix} \begin{matrix} \text{happy} \\ \text{mad} \\ \text{tired} \end{matrix} \quad \text{against a true one-hot label} \quad y = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Some deployed models expose part of their output pipeline — such as the full probability vector $\hat{y} = \text{softmax}(Z)$ — rather than just the single sampled token. The more of that pipeline an API reveals, the more an attacker has to work with. This is not an abstract concern: it is exactly how the black-box attacks of Section 8.1 are organized.

6 Threat Models

A trained ML pipeline has (at least) three parties: a *data holder* (a central server that holds D and trains $\mathcal{M}$), a *model user* (who queries $\mathcal{M}$ with an input x and receives a prediction), and a *data subject* (whose record may or may not be in D). Any privacy question about this pipeline is an instance of one template:

I want to protect the ____’s information from the ____, assuming the latter either *follows the protocol* but tries to learn extra information, or *breaks the protocol* outright.

Filling in the blanks gives genuinely different problems: protecting the data subject from the model user is a different threat model than protecting the data subject from the data holder itself, and

“honest-but-curious” (follows protocol) is a much weaker adversary than one willing to deviate from it. For the rest of this lecture, we fix one instantiation: protect the *data subject*’s information from the *model user*, who is honest-but-curious — they only ever query $\mathcal{M}$ through its intended interface, but try to learn more from the answers than the interface was meant to reveal. This is exactly the threat model membership and attribute inference assume.

7 Attacks on ML Models

Definition 7.1. A *membership inference* attack leaks information about *who* was in the training data D .

Definition 7.2. An *attribute inference* attack leaks information about an individual training sample’s attributes — not just whether it was present, but something specific about it.

The rest of this lecture develops membership inference. Attribute inference is left for later.

8 Membership Inference, Formally

Fix a training set D and a model $\mathcal{M} = \text{Train}(D)$.

Definition 8.1. The *membership inference problem* is: given a sample x and the model $\mathcal{M}$, determine whether $x \in D$.

The common approach

1. Design an algorithm $A(x) \rightarrow \{0, 1\}$, where $A(x) = 1$ is read as “ $x \in D$ ” and $A(x) = 0$ as “ $x \notin D$.”
2. Evaluate A on a labeled test set $U = (u_1, \dots, u_i, \dots, u_m)$, some of whose entries are genuine members of D and the rest of which are not.
3. Measure A ’s true and false positive rates on U : how often it correctly flags a member, and how often it incorrectly flags a non-member.

8.1 White-Box versus Black-Box Access

How much the attacker A can see about $\mathcal{M}$ splits membership inference into two regimes [2].

- **White-box.** The attacker sees every intermediate computation — the full sequence of weights $\theta_1, \dots, \theta_i$ and hidden activations $h(x; \theta_1), h(x; \theta_2), \dots$ — in addition to the final output $\hat{p}(y | x)$.
- **Black-box.** The attacker only ever sees the final output $\hat{p}(y | x)$ for a query x they choose; every intermediate weight and activation is hidden.

Black-box access itself comes in degrees, ordered by how much of the output the API reveals:

The takeaway from Section 5 recurs here in sharper form: the row of Table 1 an API exposes is exactly the parameter that controls how hard membership inference is against it.

Prediction output	Description
Full confidence scores	The attacker receives every class probability for a queried input. Because the predicted label is implicit in this vector, the attacker can also compute the input's prediction loss (e.g. cross-entropy).
Top- K confidence scores	The attacker receives only the K largest class probabilities — for instance, the top three, out of many more classes.
Prediction label only	The attacker receives only the single predicted label: the most limited access considered here.

Table 1: Black-box access levels for membership inference, ordered from most informative (top) to least (bottom); adapted from [2].

9 The Shadow-Model Attack

The best-known black-box attack [5] does not try to reason about the target model $\mathcal{M}$ directly. Instead, it builds many *shadow models* that imitate $\mathcal{M}$'s training pipeline, and learns to recognize the fingerprint that training leaves on a model's output.

The shadow-model attack

1. Assume access to *shadow data*: a pool of records drawn from a distribution similar to D , disjoint from D itself.
2. Partition the shadow data into k shadow training sets $D'_1, \dots, D'_k$, each with its own held-out shadow test set $T_1, \dots, T_k$.
3. Train k *shadow models*, one per D'_j , using the same learning algorithm and (assumed) hyperparameters as the target pipeline.
4. For each shadow model, collect its prediction vectors on its own shadow training set (labeled *member*) and on its shadow test set (labeled *non-member*).
5. Pool these labeled prediction vectors across all k shadow models, and train a binary *attack model* $g(z; \theta^*)$ that maps a prediction vector z to a member/non-member decision.
6. Apply g to the target model $\mathcal{M}$'s prediction on the actual query, as the final step.

Three practical obstacles stand between this recipe and a working attack. Realistic shadow data may simply not be available to the attacker. The target's hyperparameters — architecture, training procedure, regularization — are usually unknown, so the shadow models can only approximate the target pipeline. And training k separate shadow models is computationally expensive.

Even so, the attack works, and it works better on some inputs than others. Figure-quality results reported by the ML Privacy Meter project [3], built on the methodology of Song and Shmatikov [6], evaluate the shadow-model attack on SATED (Speaker Annotated TED talks): true positive rate against false positive rate, for inputs of increasing length. Short inputs (one sentence) give a fairly weak attack, well below the diagonal-to-corner “perfect attack” curve; as the number of sentences per example grows toward the full transcript, the ROC curve climbs toward that corner, approaching a perfect attack. Longer, more idiosyncratic inputs leave a clearer memorization fingerprint for the attack model

to pick up on.

10 How (Not) to Evaluate Membership Inference

Membership inference should not be reported as an average-case guarantee. AUC — the area under the ROC curve, averaged over every threshold — is a poor metric for it. Report true positive rate at a fixed, low false positive rate instead.

Carlini et al. [1] make this point precisely. An attack can have mediocre AUC while still being a serious privacy failure, if it identifies even a small subset of members with very high confidence and very few false alarms: that subset's privacy is genuinely broken, and no amount of poor average-case performance on the easy majority of points undoes that. Averaging across all thresholds hides exactly this. Many published attacks, evaluated only by AUC, turn out to perform poorly in the low-false-positive-rate regime that this stronger, worst-case-flavored standard demands — the regime that actually determines whether the attack constitutes a meaningful privacy violation.

Looking ahead. We now have membership inference in hand: the problem, the white-box and black-box threat models, a canonical attack, and a lesson about how to evaluate one honestly. Next time we will discuss more advanced membership inference attacks, as well as attribute inference attacks.

Key points

- **Classification reuses the regression recipe.** An architecture, a loss, and data still determine the model; only the output (a probability vector over classes, via one-hot labels) and the loss (often cross-entropy) change.
- **“Logit” means two different things.** The statistical logit is a unique real number determined by a probability; the AI logit is a vector defined only up to an additive constant, since softmax is shift-invariant.
- **What an API exposes is an attack surface.** Full confidence scores, top- K scores, and label-only access give an attacker successively less to work with for membership inference. Models that expose confidence scores (probability vectors) can leak information about a training dataset.
- **The shadow-model attack learns a fingerprint.** Many surrogate models, trained to imitate the target's pipeline, supply labeled (member/non-member) prediction vectors that train a separate attack model — no direct access to the target's training data is needed.
- **Evaluate at low FPR, not by AUC.** A membership-inference attack that reliably identifies even a small, confident subset of members is a real privacy failure, even if its average-case performance over all thresholds looks unremarkable.

Further reading

- Shokri, Stronati, Song, and Shmatikov, *Membership Inference Attacks Against Machine Learning Models* [5]: the shadow-model attack of Section 9.
- Hu, Salicic, Sun, Dobbie, Yu, and Zhang, *Membership Inference Attacks on Machine Learning: A Survey* [2]: the white-box/black-box taxonomy of Section 8.1.
- Carlini, Chien, Nasr, Song, Terzis, and Tramèr, *Membership Inference Attacks From First Principles* [1]: the evaluation critique of Section 10.
- Song and Shmatikov, *Auditing Data Provenance in Text-Generation Models* [6], and the [ML Privacy Meter](#) project: the source of the SATED results in Section 9.
- Radford, Wu, Child, Luan, Amodei, and Sutskever, *Language Models Are Unsupervised Multitask Learners* [4]: GPT-2.

Acknowledgement

The white-box/black-box comparison and the black-box access-level table are adapted from Hu et al.'s survey [2]. The shadow-model diagram is adapted from Shokri et al. [5]. The SATED ROC curves are from the ML Privacy Meter project, based on Song and Shmatikov [6]. Corrections and suggestions are welcome.

References

- [1] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *IEEE Symposium on Security and Privacy, S&P '22*, 2022.
- [2] Hongsheng Hu, Zoran Salicic, Lichao Sun, Gillian Dobbie, Philip S. Yu, and Xuyun Zhang. Membership inference attacks on machine learning: A survey. *ACM Computing Surveys*, 54(11s), 2022.
- [3] ML Privacy Meter contributors. ML Privacy Meter. https://github.com/privacytrustlab/ml_privacy_meter, 2022.
- [4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [5] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy, S&P '17*, pages 3–18, 2017.
- [6] Congzheng Song and Vitaly Shmatikov. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '19*, pages 196–206, 2019.